

members: Sanuji Gamagedara e2401855,
Yasodha Buthpitiya e2403004,
Samurdhika Susanthi e2401866

Teamwork II (Agile Software Development)

1.What is Agile software development, and why was it developed?

Agile software development is a group of software development methodologies based on iterative and incremental development, where requirements and solutions evolve through collaboration between self-organizing, cross-functional teams.

2.What are the four core values of the Agile Manifesto?

1. Individuals and interactions over processes and tools

This means people and good communication are more important than strict rules or tools.

Example: developers discuss a problem directly with each other instead of sending them through emails.

2. Working software over documentation

This means having software that actually works is more important than creating lots of documents.

Example: A team releases a basic working login feature instead of spending weeks writing detailed documentation first.

3. Customer collaboration over contract negotiation

This means working closely with the customer is more important than simply following an original agreement.

Example: A customer tests an early version of an app and suggests changes, which the development team then adds.

4. Responding to change over following a plan

This means being flexible when requirements change instead of strictly following the original plan.

Example: If users request a new payment option during development, the team adjusts the upcoming work to include it.

3.What are the most important principles of Agile development?

- Customer satisfaction by rapid delivery of useful software

deliver useful software quickly so that the customer will be satisfied.

- Welcome changing requirements, even late in development

since the customers need can change, agile can accept changes even during the development.

- Working software is delivered frequently (weeks rather than months)

so the customer can test the software frequently.

- Close, daily cooperation between businesspeople and developers

the developers and the customers has to have a regular conversations between them.

- Continuous attention to technical excellence and good design

regular checkups and improve the software and the development process.

4.Compare RUP and Agile.

CHANGES

Planning :

RUP = more planning is done at the beginning and throughout the process.

Agile = planning is done in a small cycles and can change easily.

Documentation:

RUP = more detailed documentation

Agile = less documentation

Iterations:

RUP = development is divided into phases and iterations

Agile = short iterations

Customer involment:

RUP = customer gives feedbackes at important stages

Agile = regular customer feedback

requirements:

RUP = requirements are studied and the changes are more controlled.

Agile = requirements can change easily.

SIMILARITIES

- Both use iterative development rather than building everything at once.
- Both involve regular feedback and testing.
- Both aim to produce high-quality software.
- Both recognize that requirements may need to be refined during development.

5.Compare the Spiral model and Agile development.

	Spiral Model	Agile
Risk	Focuses strongly on identifying and reducing risks at each cycle.	Reduces risk through frequent releases, testing, and customer feedback.
Change	Changes can be made, but they are usually analyzed carefully because they may affect the plan.	Welcomes change, even late in development.
Iteration	Development happens in repeated spirals, with each cycle including planning, risk analysis, development, and evaluation.	Development happens in short iterations or sprints, producing working software regularly.
Customer feedback	Customers usually review and evaluate the product during each cycle.	Customers provide frequent and continuous feedback throughout development.

Similarities

- Both use iteration instead of developing the entire system only once.
- Both allow requirements to change.
- Both use customer feedback to improve the product.
- Both help reduce project risks by identifying problems early.

6.A customer frequently changes the requirements of a software project.

Agile, because it welcomes frequent changing requirements. This will allow the team to change and improve the software easily.

7.Compare Scrum, Extreme Programming (XP), and Feature-Driven Development (FDD).

	Scrum	Extreme Programming (XP)	Feature-Driven Development (FDD)
Main focus	Managing and organizing the development process	Software quality and engineering practices	Developing small, useful features
Work style	Work is divided into short Sprints	Work is done in short iterations with continuous testing	Development is organized around specific features
Planning	Sprint planning and product backlog	Simple planning with frequent changes	Detailed feature planning and design
Customer involvement	Regular feedback from the customer/Product Owner	Very high customer involvement	Customer requirements guide feature development
Documentation	Moderate	Usually minimal	More structured documentation
Key practices	Daily Scrum, Sprint Review, Retrospective, Product Backlog	Pair programming, Test-Driven Development (TDD), continuous integration, refactoring	Develop overall model, create feature list, design and build by feature
Team size	Usually small, cross-functional teams	Small, highly collaborative teams	Often suitable for larger development teams
Flexibility	High	Very high	Moderate to high

	Scrum	Extreme Programming (XP)	Feature-Driven Development (FDD)
Best suited for	Projects where priorities change frequently	Projects where code quality and frequent feedback are very important	Large projects with many clearly defined features

8.What are the main advantages and disadvantages of Scrum?

Advantages of Scrum

Flexible: Changes can be made between sprints.

Fast delivery: Working software is delivered regularly.

Customer feedback: Customers can give feedback frequently.

Better visibility: Progress is easy to track using the product backlog and sprint goals.

Continuous improvement: Teams review their work and improve after each sprint.

Disadvantages of Scrum

Requires teamwork: It may not work well if team members do not communicate.

Can be difficult to manage: Regular meetings and sprint planning require time.

Needs experienced team members: Teams need to understand Scrum practices.

Less suitable for fixed plans: It can be difficult when requirements, budget, and deadlines cannot change.

Scope can change frequently: Too many changes can make it difficult to maintain focus.

When Scrum Works Well

Scrum is a good choice when:

- Requirements are likely to change frequently.
- The customer wants to give regular feedback.
- The project can be divided into small pieces of working software.
- The team is small, collaborative, and self-organizing.
- The project benefits from frequent releases and improvements.

Example: Developing a mobile app where new features are added based on user feedback.

When Scrum May Not Work Well

Scrum may not be suitable when:

- Requirements are completely fixed from the beginning.
- The project has strict regulations requiring extensive documentation and fixed processes.
- The team cannot communicate or work together regularly.
- The project cannot be divided into smaller deliverable pieces.
- There is no opportunity for regular customer or stakeholder feedback.

Example: A project with a fixed contract where every requirement, deadline, and deliverable must remain unchanged.

9.Explain the three main Scrum roles: Product Owner, Scrum Master, and Developers

- **Scrum Master:** Maintains the processes
- **Product Owner:** who represents the stakeholders and the business
- **Team:** a cross-functional group who do the actual analysis, design, implementation, testing, etc.

10.What are Scrum artifacts and events?

1. Artifacts:

- **Product Backlog:** A prioritized list of all the features, bug fixes, and improvements that need to be made to the product.
- **Sprint Backlog:** A subset of the product backlog that the team commits to completing during a sprint.
- **Increment:** The sum of all completed backlog items during a sprint, representing the current state of the product.

2. Events:

- **Sprint:** A time-boxed iteration, typically lasting 2-4 weeks, during which the team works on a specific set of tasks from the sprint backlog.
- **Sprint Planning:** A meeting where the team plans the work for the upcoming sprint.

- **Daily Stand-up (Daily Scrum):** A short, daily meeting where team members discuss what they did yesterday, what they plan to do today, and any blockers they are facing.
- **Sprint Review:** A meeting at the end of the sprint where the team demonstrates the work completed to stakeholders.
- **Sprint Retrospective:** A meeting where the team reflects on the sprint and discusses ways to improve in the next iteration.

11. Imagine that your team is developing a mobile application using Scrum.

Design one Sprint for the project, including the Sprint Goal, some Product Backlog items, Sprint activities, and expected Increment.

Project: Food Delivery Mobile App

Sprint length: 2 weeks

	Description
Sprint Goal	Create a basic ordering system that allows users to browse restaurants, select food, and place an order. 1. User registration and login 2. Display restaurant list
Product Backlog Items	3. Display food menu 4. Add food to cart 5. Place an order
Sprint Activities	Days 1–2: Design screens and set up the database. Days 3–5: Develop login and restaurant list. Days 6–8: Develop menu and shopping cart. Days 9–10: Implement ordering, testing, and fixing bugs.
Expected Increment	A working version of the mobile app where a user can log in, browse restaurants and menus and order food.

12. What are the main roles and practices in Extreme Programming (XP)?

main roles: developer, customer, team

he key practices of XP are:

1. **Pair Programming:** Developers work in pairs, with one writing the code while the other reviews it in real-time. This practice improves code quality and facilitates knowledge sharing.
2. **Test-Driven Development (TDD):** Tests are written before the actual code. This ensures that the code meets the requirements and helps catch bugs early.
3. **Continuous Integration:** Code changes are integrated into the main codebase frequently, often multiple times a day. This practice helps detect integration issues early.
4. **Refactoring:** Developers continuously improve the codebase by restructuring existing code without changing its behavior, making it cleaner and more maintainable.

13.What are the main advantages and disadvantages of XP?

Advantages:

- High-Quality Code emphasizes testing and continuous integration, leading to fewer defects.
- Adaptability allows teams to quickly adapt to changing requirements and customer needs.
- Increased Collaboration encourages teamwork and communication, resulting in a more cohesive development process.

Disadvantages:

- Cultural Shift: Requires a change in mindset and practices, which can be challenging for teams used to traditional methodologies.
- Requires Discipline: Practices like pair programming and TDD require consistent application and discipline from the team.

14.What is Feature-Driven Development (FDD)?

Feature Driven Development (FDD) is an agile software development methodology that focuses on delivering tangible, working software features within short iterations.

The following five tasks are involved in the whole process:

1. Build the overall model
2. Specify the feature list
3. Make plans for the individual tasks
4. Feature design
5. Implementation

15.Case study: Your team must develop a large online shopping system. Requirements are expected to change frequently, the customer wants regular demonstrations, and the system must be delivered incrementally. **Which development model or combination of models would you choose—Scrum, XP, FDD, RUP, Spiral, or another approach?** Justify your choice and explain how your team would organize the development process.

we would choose Scrum with XP practices for developing the large online shopping system.

Why Scrum?

Scrum is suitable because the requirements can change frequently. The project can be divided into short Sprints. The customer can see a working part of the system at the end of each Sprint and give feedback.

Why XP practices?

XP can be used to improve software quality. The team can use pair programming, test-driven development (TDD), continuous integration, and refactoring. These practices help find errors early and keep the code easy to maintain.

Development Process

Create and prioritize the Product Backlog with the customer.

Select important features for each Sprint.

Develop features using XP practices.

Use Daily Scrum meetings to discuss progress and problems.

At the end of each Sprint, demonstrate the working software to the customer.

Collect customer feedback and update the Product Backlog.

Use the next Sprint to develop new features or improve existing ones.